

Tarea-Examen 3: Simulación Estocástica

José Jorge Martínez de la Cruz

26 de Mayo 2026

```
library(ggplot2)
set.seed(42)
```

Problema 1

Sea

$$\pi = (0.1, 0.2, 0.3, 0.4)$$

la distribución objetivo, con soporte en el conjunto $\{1, 2, 3, 4\}$. Partiendo de algún valor inicial $x_0 \in \{1, 2, 3, 4\}$, genera 120 valores mediante el algoritmo de Metropolis-Hastings usando como función de propuesta la matriz

$$Q = \begin{pmatrix} 1/4 & 1/4 & 1/4 & 1/4 \\ 1/4 & 1/4 & 1/4 & 1/4 \\ 1/4 & 1/4 & 1/4 & 1/4 \\ 1/4 & 1/4 & 1/4 & 1/4 \end{pmatrix}.$$

Además, encuentra explícitamente la matriz de transición $P = (P_{ij})$, y justifica por qué la cadena es convergente y compara los histogramas: $x_1, \dots, x_{40}$, $x_{41}, \dots, x_{80}$ y $x_{81}, \dots, x_{120}$.

Solución

Para realizar este ejercicio, recordemos que el algoritmo de Metropolis-Hastings permite construir una cadena de Markov cuya distribución estacionaria sea una distribución objetivo dada. En este problema, la distribución objetivo es discreta y está dada por:

$$\pi(1) = 0.1, \quad \pi(2) = 0.2, \quad \pi(3) = 0.3, \quad \pi(4) = 0.4.$$

La matriz de propuesta Q tiene probabilidades de transición uniformes, es decir, desde cualquier estado actual i , nos vamos a cualquiera de los estados $j \in \{1, 2, 3, 4\}$ con probabilidad

$$q_{ij} = \frac{1}{4}.$$

También notemos que $q_{ij} = q_{ji}$, eso quiere decir que Q es una matriz simétrica así, la probabilidad de aceptación de Metropolis-Hastings se simplifica a

$$\alpha(i, j) = \min \left\{ 1, \frac{\pi(j)}{\pi(i)} \right\}.$$

Esto significa que si el estado j tiene mayor probabilidad objetivo que el estado actual i , entonces se acepta automáticamente. En cambio, si el estado tiene menor probabilidad objetivo, se acepta con probabilidad $\pi(j)/\pi(i)$.

El algoritmo que usaremos es el siguiente:

1. Se fija un valor inicial $x_0 \in \{1, 2, 3, 4\}$.
2. Si la cadena está en el estado actual i , se propone un estado j usando la distribución uniforme en $\{1, 2, 3, 4\}$.
3. Se calcula

$$\alpha(i, j) = \min \left\{ 1, \frac{\pi(j)}{\pi(i)} \right\}.$$

4. Se genera $U \sim \text{Uniforme}(0, 1)$. Si $U < \alpha(i, j)$, se acepta el candidato y $x_t = j$. Si no, se rechaza y $x_t = i$.
5. Se repite el procedimiento hasta obtener 120 valores.

En el código siguiente usamos la semilla 42 y tomamos $x_0 = 1$:

```
# semilla
set.seed(42)

# Distribución objetivo
pi_obj <- c(0.1, 0.2, 0.3, 0.4)

# Soporte de la distribución objetivo
soporte <- 1:4

# Matriz de propuesta Q: desde cualquier estado se propone Uniforme{1,2,3,4}
Q <- matrix(1/4, nrow = 4, ncol = 4)
rownames(Q) <- paste0("i=", soporte)
colnames(Q) <- paste0("j=", soporte) #Nombres

n <- 120

# Valor inicial x_0
x_actual <- 1

# Vector donde guardaremos x_1, ..., x_120
valores <- numeric(n)

for (t in 1:n) {
  # Propuesta: como Q es uniforme, elegimos j con probabilidad 1/4
  j <- sample(x = soporte, size = 1, prob = Q[x_actual, ])

  # Probabilidad de aceptación de Metropolis-Hastings
  alpha_ij <- min(1, pi_obj[j] / pi_obj[x_actual])

  # Paso de aceptación/rechazo
  if (runif(1) < alpha_ij) {
    x_actual <- j
  }

  # Guardamos el nuevo estado de la cadena
  valores[t] <- x_actual
}
```

```

valores[t] <- x_actual
}

# Mostramos los 120 valores generados
valores

##      [1] 1 3 4 4 4 3 1 3 1 3 1 1 2 3 3 4 3 3 2 2 3 3 3 3 3 3 3 3 4 3 4 4 1 3 4 2 2
##     [38] 2 2 4 4 3 3 2 2 4 4 4 3 4 4 2 2 4 4 4 4 2 4 3 4 4 4 4 4 4 4 4 1 1 2 3 4 4
##     [75] 3 4 4 4 2 2 2 4 4 3 2 4 3 3 3 3 3 4 4 2 2 2 2 4 4 4 4 2 1 3 4 3 3 4 4 3
##    [112] 4 4 3 2 3 3 3 3 4

```

Matriz de transición explícita

Ahora construiremos explícitamente la matriz de transición $P = (P_{ij})$ de la cadena de Markov inducida por Metropolis-Hastings.

Para $i \neq j$, la probabilidad de pasar de i a j está dada por:

$$P_{ij} = q_{ij}\alpha(i, j).$$

Como $q_{ij} = 1/4$, entonces

$$P_{ij} = \frac{1}{4} \min \left\{ 1, \frac{\pi(j)}{\pi(i)} \right\}, \quad i \neq j.$$

La probabilidad diagonal P_{ii} incluye dos posibilidades: que se proponga el mismo estado i , o que se proponga otro estado y sea rechazado. Por tanto,

$$P_{ii} = q_{ii} + \sum_{j \neq i} q_{ij}(1 - \alpha(i, j)).$$

Calculando entrada por entrada obtenemos:

$$P = \begin{pmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{8} & \frac{3}{8} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{12} & \frac{1}{6} & \frac{1}{2} & \frac{1}{4} \\ \frac{1}{16} & \frac{1}{8} & \frac{3}{16} & \frac{5}{8} \end{pmatrix}.$$

La cadena es convergente porque cumple las condiciones estándar para una cadena finita, irreducible y aperiódica.

Primero, el espacio de estados es finito:

$$S = \{1, 2, 3, 4\}.$$

Segundo, la cadena es irreducible. En efecto, para cualesquiera $i, j \in S$, con $i \neq j$, se tiene

$$P_{ij} = \frac{1}{4} \min \left\{ 1, \frac{\pi(j)}{\pi(i)} \right\} > 0,$$

porque todas las entradas de π son estrictamente positivas. Por tanto, desde cualquier estado se puede llegar a cualquier otro estado en un solo paso con probabilidad positiva.

Tercero, la cadena es aperiódica. Pues

$$P_{11} = \frac{1}{4}, \quad P_{22} = \frac{3}{8}, \quad P_{33} = \frac{1}{2}, \quad P_{44} = \frac{5}{8},$$

de modo que cada estado tiene probabilidad positiva de permanecer en sí mismo. Esto implica que el periodo de cada estado es 1.

Además, por la construcción de Metropolis-Hastings, la cadena satisface balance detallado:

$$\pi(i)P_{ij} = \pi(j)P_{ji}, \quad i, j \in S.$$

Por lo tanto, π es estacionaria. Como la cadena es finita, irreducible y aperiódica, se concluye que

$$\lim_{n \rightarrow \infty} P_{ij}^n = \pi(j),$$

para todo $i, j \in S$. Es decir, la distribución de la cadena converge a la distribución objetivo.

Ahora dividimos la muestra generada en tres bloques de tamaño 40:

$$\{x_1, \dots, x_{40}\}, \quad \{x_{41}, \dots, x_{80}\}, \quad \{x_{81}, \dots, x_{120}\}.$$

Como cada bloque tiene tamaño 40, las frecuencias esperadas bajo la distribución objetivo son:

$$40\pi = 40(0.1, 0.2, 0.3, 0.4) = (4, 8, 12, 16).$$

Es decir, si un bloque se parece mucho a la distribución objetivo, esperaríamos observar aproximadamente 4 valores iguales a 1, 8 valores iguales a 2, 12 valores iguales a 3 y 16 valores iguales a 4.

```
# Construimos un dataframe con los valores simulados y su bloque correspondiente
datos_mh <- data.frame(
  iteracion = 1:n,
  valor = factor(valores, levels = soporte),
  bloque = rep(c("x1 a x40", "x41 a x80", "x81 a x120"), each = 40)
)
```

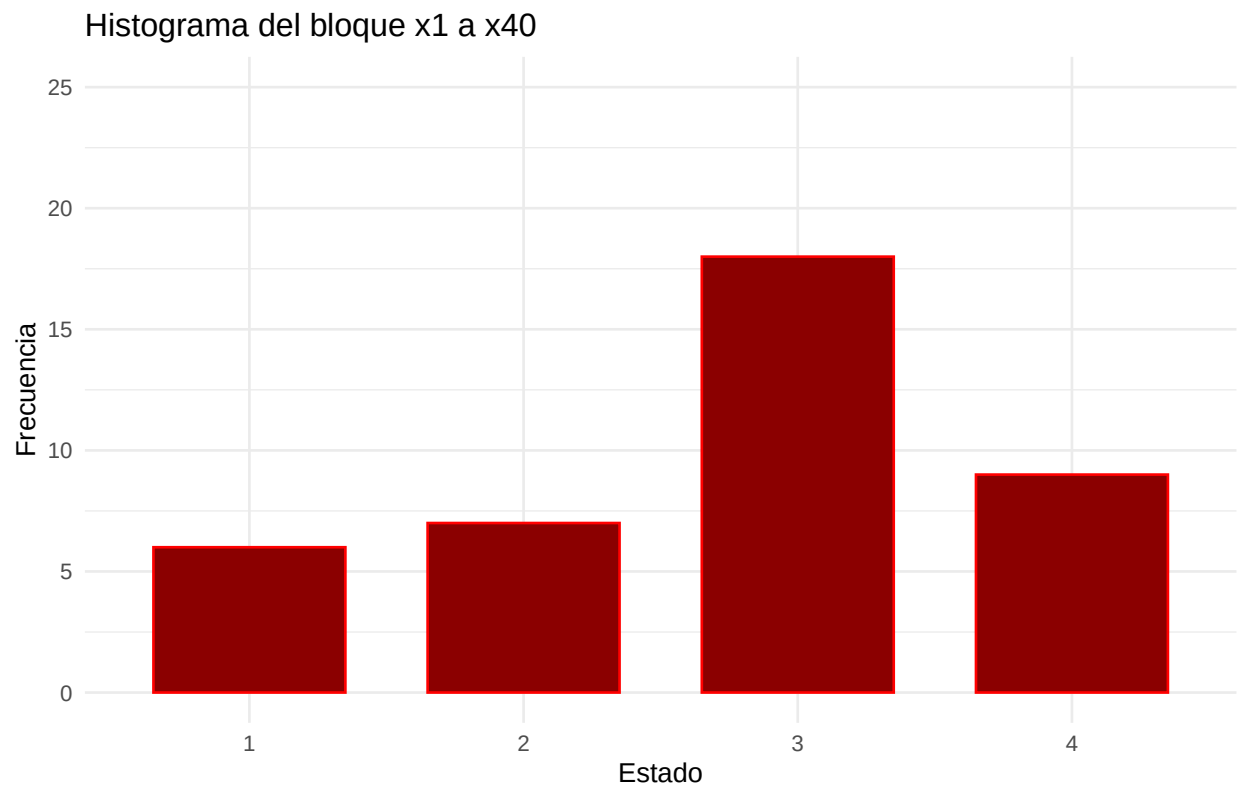
```
# Función auxiliar para graficar cada bloque
histograma_bloque <- function(nombre_bloque) {
  datos_bloque <- subset(datos_mh, bloque == nombre_bloque)

  ggplot(datos_bloque, aes(x = valor)) +
    geom_bar(fill = "darkred", color = "red", width = 0.7) +

    scale_y_continuous(breaks = seq(0, 25, by = 5), limits = c(0, 25)) +
    labs(
      title = paste("Histograma del bloque", nombre_bloque),
      x = "Estado",
      y = "Frecuencia"
    ) +
    theme_minimal()
}
```

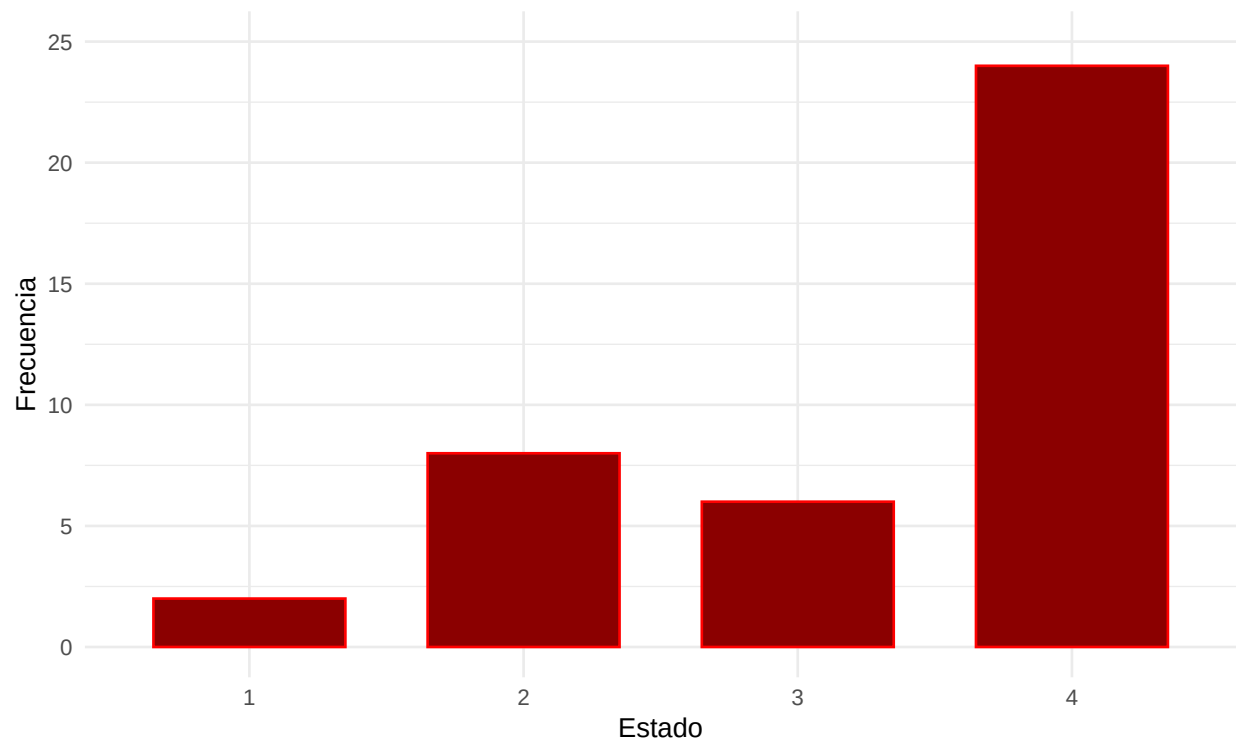
```
}
```

```
histograma_bloque("x1 a x40")
```



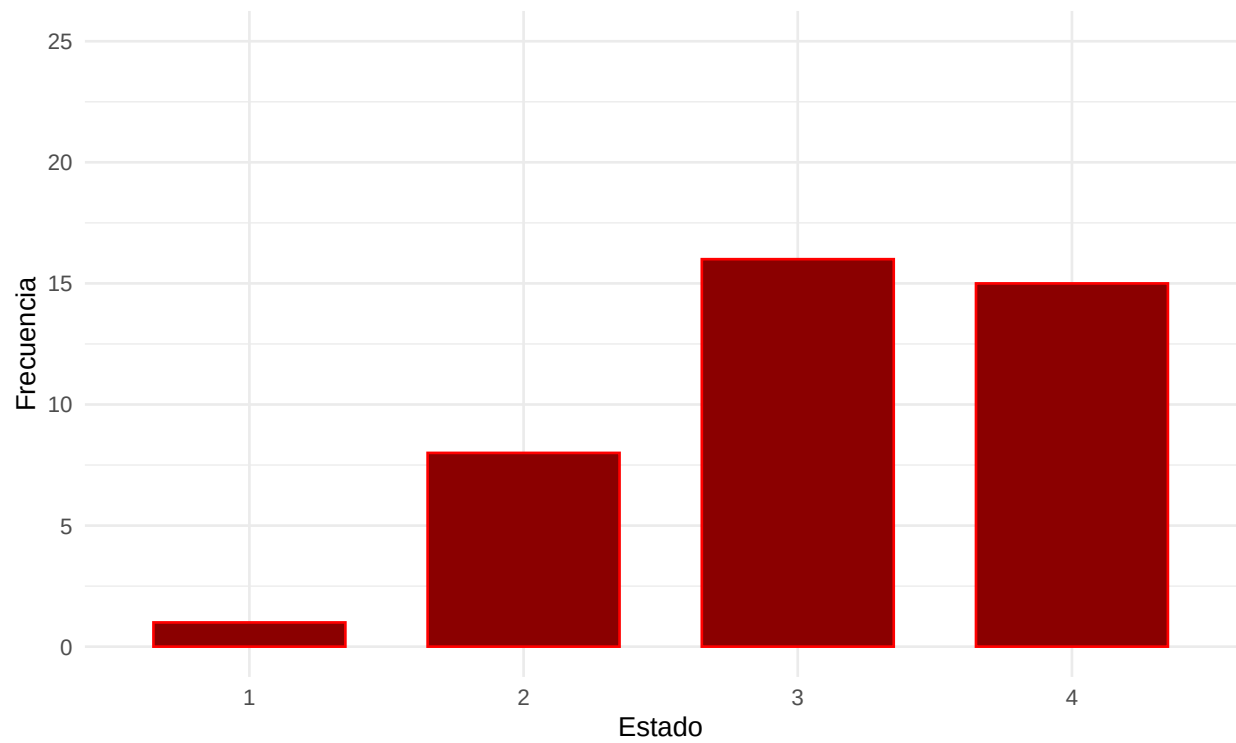
```
histograma_bloque("x41 a x80")
```

Histograma del bloque x41 a x80



```
histograma_bloque("x81 a x120")
```

Histograma del bloque x81 a x120



Al comparar los tres histogramas, podemos notar que las observaciones más tardías se parecen más a π , porque la cadena ha tenido más tiempo para olvidar el valor inicial x_0 . Esto correspondería al periodo del calentamiento. No obstante, como cada bloque tiene solamente 40 observaciones, todavía puede haber variabilidad aleatoria considerable. Por esa razón, la última muestra puede verse mejor que las anteriores, pero es perfecta.

Problema 2

Considere el conjunto de datos **Mediciones**, el cual contiene registros bivariados de humedad y temperatura. Se observa que existen valores faltantes en la variable de temperatura; no obstante, se sabe por el mecanismo de recolección que los registros correspondientes a los 5 días con mayor temperatura están completos dentro del conjunto.

Construya un algoritmo **EM** para imputar los datos faltantes de temperatura.

Solución

Sea

$$X_1 = \text{humedad}, \quad X_2 = \text{temperatura}.$$

Supondremos que el vector aleatorio bivariado

$$X = \begin{pmatrix} X_1 \\ X_2 \end{pmatrix}$$

puede aproximarse mediante una distribución normal bivariada:

$$X \sim N_2(\mu, \Sigma),$$

donde

$$\mu = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{21} & \sigma_{22} \end{pmatrix}.$$

En este problema la humedad está observada para todos los registros, pero la temperatura tiene valores faltantes. Por lo tanto, para una fila con temperatura faltante, observamos $X_1 = x_1$ y queremos imputar X_2 .

Bajo normalidad bivariada, la distribución condicional de X_2 dado $X_1 = x_1$ es normal, y su esperanza condicional está dada por:

$$\mathbb{E}(X_2 \mid X_1 = x_1) = \mu_2 + \frac{\sigma_{21}}{\sigma_{11}}(x_1 - \mu_1).$$

Además, la varianza condicional está dada por:

$$\text{Var}(X_2 \mid X_1) = \sigma_{22} - \frac{\sigma_{21}^2}{\sigma_{11}}.$$

El algoritmo EM no solo debe reemplazar cada dato faltante por su esperanza condicional. También debe conservar la varianza asociada a esos valores faltantes para no subestimar la varianza.

```
mediciones <- read.csv("Mediciones.csv")
```

```
# Revisamos las primeras observaciones  
head(mediciones)
```

```
##   humedad temperatura  
## 1   36.70         53.5  
## 2   37.00         51.0  
## 3   39.30         39.0  
## 4   38.75         42.0  
## 5   40.45         47.0  
## 6   37.30         44.5
```

```
#Número de datos faltantes  
colSums(is.na(mediciones))
```

```
##   humedad temperatura  
##           0          20
```

```
# Renombramos
```

```
X1_humedad <- mediciones$humedad  
X2_temperatura_obs <- mediciones$temperatura
```

```
n <- length(X1_humedad)  
indices_faltantes <- which(is.na(X2_temperatura_obs))  
indices_observados <- which(!is.na(X2_temperatura_obs))
```

```
# Verificamos los 5 valores de temperatura observados.
```

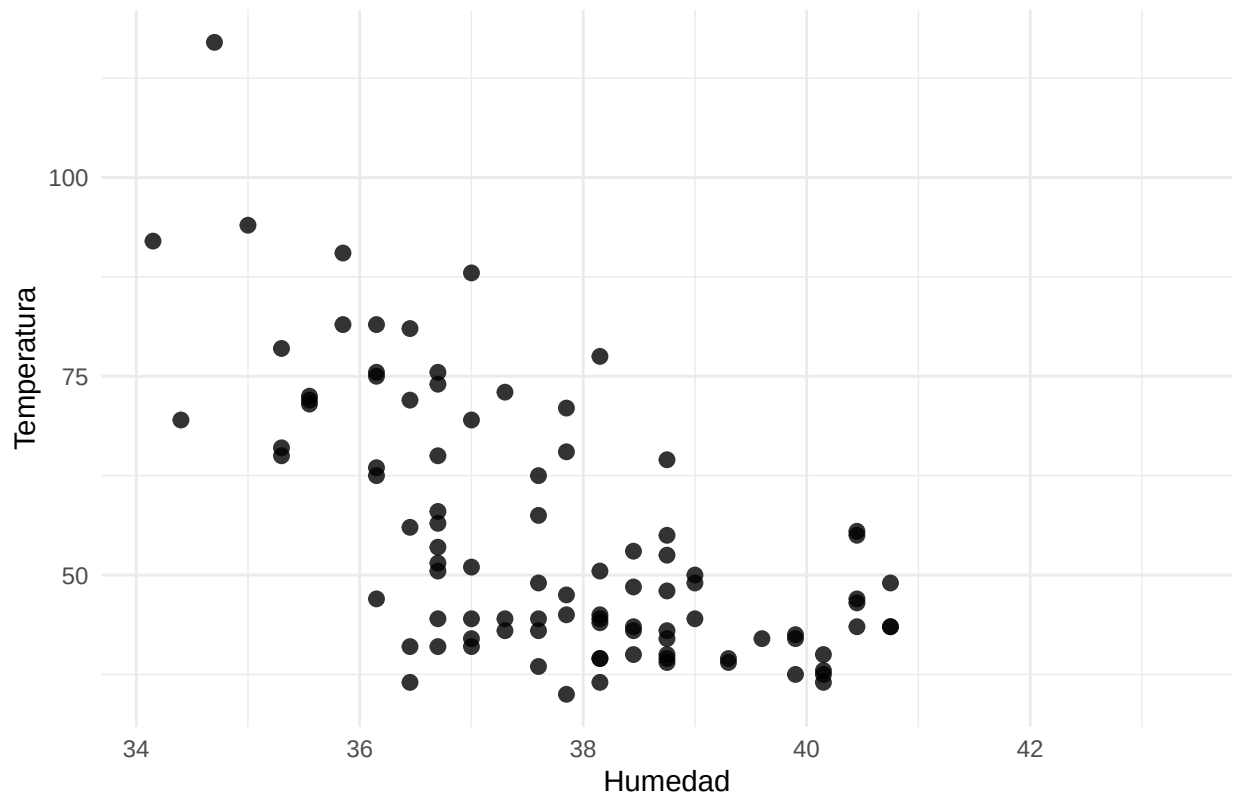
```
orden_top_temp <- order(mediciones$temperatura, decreasing = TRUE, na.last = NA) #Usamos order para ord  
top_5_temp <- mediciones[orden_top_temp[1:5], ]
```

```
top_5_temp
```

```
##   humedad temperatura  
## 82   34.70         117.0  
## 68   35.00          94.0  
## 85   34.15          92.0  
## 25   35.85          90.5  
## 70   37.00          88.0
```

```
ggplot(mediciones, aes(x = humedad, y = temperatura)) +  
  geom_point(size = 2.3, alpha = 0.8, na.rm = TRUE) +  
  labs(  
    title = "Relación observada entre humedad y temperatura",  
    x = "Humedad",  
    y = "Temperatura"  ) +  
  theme_minimal()
```


Relación observada entre humedad y temperatura



Calculemos la correlación para el paso **E**, la imputación por esperanza condicional necesita la relación lineal entre humedad y temperatura inducida por el modelo normal bivariado.

```
# Correlación usando únicamente casos completos
cor_completa <- cor(X1_humedad[indices_observados],
                    X2_temperatura_obs[indices_observados])
```

El algoritmo EM alterna dos pasos:

1. **Paso E**: calcular el valor esperado de las temperaturas faltantes dadas las humedades observadas y los parámetros actuales.
2. **Paso M**: actualizar los estimadores de μ y Σ usando los datos completados, pero agregando la corrección de varianza condicional para las temperaturas faltantes.

Tomamos como valores iniciales:

$$\mu^{(0)} = \begin{pmatrix} \bar{X}_1 \\ \bar{X}_{2,obs} \end{pmatrix},$$

es decir, la media de humedad con todos los datos y la media de temperatura con los datos observados. Para la matriz de covarianzas inicial usamos solamente los casos completos.

```
# Estimadores iniciales
# Matriz observable: humedad completa, temperatura con NA
datos_observados <- cbind(X1_humedad, X2_temperatura_obs)
```

```
# Inicialización con casos observados / completos
mu_est <- c(mean(X1_humedad), mean(X2_temperatura_obs, na.rm = TRUE))
sigma_est <- cov(datos_observados, use = "complete.obs") * (nrow(datos_observados[complete.cases(datos_

mu_est
```

```
## [1] 37.93103 54.55208
```

```
sigma_est
```

```
##                X1_humedad X2_temperatura_obs
## X1_humedad      2.560399      -16.85265
## X2_temperatura_obs -16.852648      265.67437
```

```
# Parámetros de control del bucle
tolerancia <- 1e-5
diferencia <- 1
iteracion <- 0
max_iter <- 500

# Guardamos la trayectoria de convergencia para graficarla después
historial <- data.frame(
  iteracion = integer(),
  diferencia = numeric(),
  mu_humedad = numeric(),
  mu_temperatura = numeric(),
  var_humedad = numeric(),
  cov_hum_temp = numeric(),
  var_temperatura = numeric()
)

# Empezamos con una copia de la temperatura observada
temperatura_imputada <- X2_temperatura_obs

while (diferencia > tolerancia && iteracion < max_iter) {
  iteracion <- iteracion + 1

  # Guardamos los parámetros anteriores para medir convergencia
  mu_ant <- mu_est
  sigma_ant <- sigma_est

  # -----
  # PASO E (Expectation):
  # Calculamos el valor esperado de los datos faltantes dados los datos
  # observados y los parámetros actuales ( $\theta^t$ ).
  # Para una Normal Bivariada, la distribución condicional  $X2 | X1$  es normal.
  # -----
  # Calculamos  $E[X2 | X1]$  para cada temperatura faltante.
```

```

temperatura_imputada <- X2_temperatura_obs

beta_condicional <- sigma_est[2, 1] / sigma_est[1, 1]

for (i in indices_faltantes) {
  temperatura_imputada[i] <- mu_est[2] + beta_condicional * (X1_humedad[i] - mu_est[1])
}

# Varianza condicional Var(X2 | X1)
var_condicional <- sigma_est[2, 2] - (sigma_est[2, 1]^2 / sigma_est[1, 1])

# -----
# PASO M (Maximization):
# Actualizamos los parámetros (mu, Sigma) maximizando la log-verosimilitud
# esperada, usando nuestros datos "completados" estadísticamente.
# -----
# Actualizamos la media usando los datos completados.
mu_est <- c(mean(X1_humedad), mean(temperatura_imputada))

# Actualizamos la matriz de covarianza
datos_imputados_temp <- cbind(X1_humedad, temperatura_imputada)
colnames(datos_imputados_temp) <- c("humedad", "temperatura")

sigma_est <- cov(datos_imputados_temp) * (n - 1) / n

# Corrección EM: agregamos la incertidumbre de los datos faltantes
# a la entrada correspondiente a Var(temperatura).
proporcion_faltante <- length(indices_faltantes) / n
sigma_est[2, 2] <- sigma_est[2, 2] + proporcion_faltante * var_condicional

# -----
# EVALUACIÓN DE CONVERGENCIA
# Medimos la distancia entre los parámetros actuales y los anteriores
# -----
diferencia <- sum((mu_est - mu_ant)^2) + sum((sigma_est - sigma_ant)^2)

historial <- rbind(
  historial,
  data.frame(
    iteracion = iteracion,
    diferencia = diferencia,
    mu_humedad = mu_est[1],
    mu_temperatura = mu_est[2],
    var_humedad = sigma_est[1, 1],
    cov_hum_temp = sigma_est[1, 2],
    var_temperatura = sigma_est[2, 2]
  )
)
}

cat("Convergencia alcanzada en", iteracion, "iteraciones.\n")

```

Bucle EM

```
## Convergencia alcanzada en 9 iteraciones.
```

```
cat("Diferencia final:", diferencia, "\n\n")
```

```
## Diferencia final: 4.036085e-06
```

```
cat("Vector de medias estimado:\n")
```

```
## Vector de medias estimado:
```

```
print(round(mu_est, 4))
```

```
## [1] 37.9310 53.3331
```

```
cat("\nMatriz de covarianzas estimada:\n")
```

```
##
```

```
## Matriz de covarianzas estimada:
```

```
print(round(sigma_est, 4))
```

```
##          humedad temperatura
```

```
## humedad      3.1674    -20.8476
```

```
## temperatura -20.8476    291.9686
```

```
# Dataset final con imputaciones
```

```
mediciones_imputadas <- mediciones
```

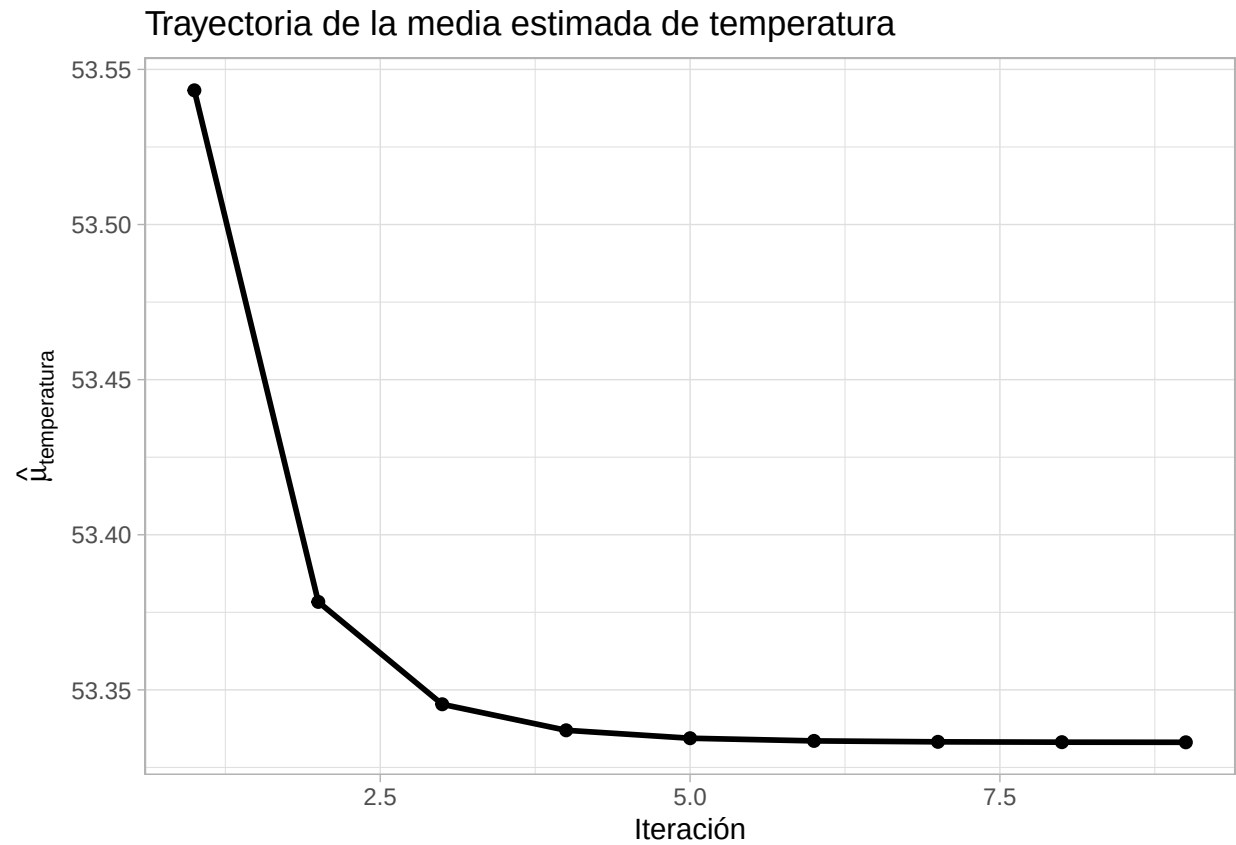
```
mediciones_imputadas$temperatura_imputada <- temperatura_imputada
```

```
mediciones_imputadas$tipo <- ifelse(is.na(mediciones$temperatura),  
                                   "Imputada por EM", "Observada")
```

```
# Para dejar una sola columna final de temperatura completa
```

```
mediciones_imputadas$temperatura_final <- ifelse(  
  is.na(mediciones_imputadas$temperatura),  
  mediciones_imputadas$temperatura_imputada,  
  mediciones_imputadas$temperatura  
)
```

```
ggplot(historial, aes(x = iteracion, y = mu_temperatura)) +  
  geom_line(linewidth = 1) +  
  geom_point(size = 1.8) +  
  labs(  
    title = "Trayectoria de la media estimada de temperatura",  
    x = "Iteración",  
    y = expression(hat(mu)[temperatura])  
  ) +  
  theme_light()
```



```
ggplot(mediciones_imputadas, aes(x = humedad, y = temperatura_final, color = tipo)) +  
  geom_point(size = 2.4, alpha = 0.85) +  
  labs(  
    title = "Datos completos después de imputar temperatura con EM",  
    x = "Humedad",  
    y = "Temperatura final",  
    color = "Tipo de dato"  
  ) +  
  theme_light()
```